DAY_01

Date : 26.08.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Write a program to calculate the total bill amount after applying a discount based
   on the purchase amount.
2. Write a program to find the factorial of a number using a loop.
3. Write a program to check whether a number is an Armstrong number.
4. Write a program to generate the Fibonacci series up to N terms.
5. Print different patterns of asterisk (*) using loops (e.g. triangle of *)
6. Write a program to find principal diagonal sum of a matrix.

==========

DAY_02

Date : 27.08.2026

Time : 5:30 P.M. to 8:00 P.M.

1. **Reverse a Number**
   Write a program to reverse the digits of a given number using a loop.
   Example: 12345 → 54321
2. **Check Palindrome Number**
   Write a program to check whether a given number is a palindrome.
   Example: 121 → Palindrome, 123 → Not Palindrome
3. **Check Prime Number**
   Write a program to check whether a given number is prime or not.
4. **Print Prime Numbers in a Range**
   Write a program to accept two numbers and print all prime numbers between them.
5. **Find Largest and Smallest Element in an Array**
   Write a program to accept N integers into an array and find the largest and smallest elements without using built-in sorting methods.
6. **Calculate Sum and Average of Array Elements**
   Write a program to accept N numbers into an array and calculate their total sum and average.

IET PUNE

7. **Count Even and Odd Numbers in an Array**
   Write a program to accept an array of integers and count how many elements are even and how many are odd.
8. **Transpose of a Matrix**
   Write a program to accept a matrix and print its transpose.
   Example: Rows become columns and columns become rows.
9. **Find Sum of Each Row and Column in a Matrix**
   Write a program to accept a matrix and calculate the sum of every individual row and every individual column.
10. **Count Frequency of Each Element in an Array**
    Write a program to accept N integers into an array and find how many times each element occurs.
    Example: 2 3 2 5 3 2 → 2: 3 times, 3: 2 times, 5: 1 time

==========

DAY_03

Date : 29.08.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Design a simple application (ex. Student Management System) using:
   i. Classes and constructors
   ii. Object references
   iii. Association between classes
   iv. Display students details
2. Print Pascal's triangle.
3. Print diamond star pattern.
4. Print butterfly pattern.

==========

IET PUNE

DAY_04

Date : 31.08.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Demonstrate boxing and unboxing
2. Demonstrate autoboxing and auto-unboxing
3. Use wrapper utility methods such as:
   i. parseInt()
   ii. parseDouble()
   iii. valueOf()
   iv. toString()
   v. compareTo()
4. Create String objects using:
   i. String literals
   ii. new String()
5. Compare strings using:
   i. ==
   ii. equals()
6. Observe the behavior of String Constant Pool.
7. Perform operations using:
   i. length()
   ii. charAt()
   iii. substring()
   iv. indexOf()
   v. contains()
   vi. replace()
   vii. split()
   viii. toUpperCase()
   ix. toLowerCase()
   x. trim()

8. Create programs demonstrating:
   i. append()
   ii. insert()
   iii. delete()
   iv. replace()
   v. reverse()
9. Compare the behavior of String, StringBuffer, and StringBuilder.

==========

IET, PUNE

DAY_05
Date : 01.09.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Design a Smart Device Management System where:
   i. Device is an abstract class
   ii. Interfaces such as Connectable, Rechargeable, and Upgradeable define device capabilities
   iii. Different devices implement one or more interfaces
   iv. Use default methods for common functionality
   v. Study when to prefer an abstract class over an interface

2. Write a program to check number is positive , negative or zero.
3. Write a program to reverse given String.
4. Write a program to find out duplicate character.
5. Write a program to count special character in a String.

==========

DAY_06
Date : 02.09.2026

Time : 5:30 P.M. to 8:00 P.M.

1. **Enumeration – Employee Department**

Create an enum named Department with the values HR, IT, FINANCE, SALES, and MARKETING.
Create an Employee class containing employee ID, name, and department.
Write a Java program to:

- Create employee objects.
- Display their details.
- Use the enum to display the department.
- Use switch to display a suitable message for each department.

---

2. **Regular Inner Class – Bank Account**

Create a class BankAccount containing account number, account holder name, and balance.
Create a **regular inner class** named AccountDetails that displays the account information by accessing the private members of BankAccount.
Write a program to demonstrate:

- Creation of the outer class object.
- Creation of the inner class object.
- Accessing outer-class private data through the inner class.

---

3. **Method-Local Inner Class – Student Grade**

Create a class Student containing student name, roll number, and marks.
Define a method calculateGrade() inside the Student class. Within this method, create a **method-local inner class** named Grade that determines the student's grade based on the marks:

- 90–100 → A
- 75–89 → B
- 60–74 → C
- 40–59 → D
- Below 40 → F

Display the student's details and grade.

---

4. **Anonymous Inner Class – Shape**

Create an interface Shape containing a method calculateArea().
Write a Java program using an **anonymous inner class** to implement the Shape interface for calculating the area of a rectangle.
Accept the length and breadth from the user and display the calculated area.

---

5. **Static Nested Class – Company and Employee**

Create a class Company containing company name and location.
Create a **static nested class** named Employee containing employee ID and employee name.
Write a Java program to:

- Create an object of the static nested Employee class without creating an object of Company.
- Display employee details along with company information.
- Demonstrate the difference between accessing static and non-static members of the outer class.

==========

DAY_07
Date : 03.09.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Develop a Employee Management System that:
   i. Uses pattern matching to process different employee objects.

2. Design and implement a **Java-based Event Management System** using the java.time.LocalDate class.
   The system should allow users to:

- Create an event by entering its **name, start date, and end date**.
- Determine the **duration of the event** in days.
- Check whether an event is **upcoming, ongoing, or completed** based on the current date.
- Identify whether two events **overlap** with each other.
- Find the **earliest and latest event** from a collection of events.
- Calculate the number of days remaining until an upcoming event.
- Display all events scheduled within a specified date range.
- Handle invalid date ranges appropriately.

3. Online Banking System

Develop a Java program for an **Online Banking System** that allows users to withdraw and transfer money.
The program should handle the following exceptions:

- Attempting to withdraw an amount greater than the available balance.
- Entering a negative or zero amount.
- Entering an invalid account number.
- Attempting to transfer money when the account has insufficient funds.

Create and use **custom exceptions** where appropriate.

---

4. Student Examination and Result System

Develop a Java application that accepts **student marks for multiple subjects** and calculates the total, average, and grade.
The program should handle:

- Invalid marks such as values below 0 or above 100.
- Non-numeric input entered by the user.
- Division-by-zero errors when calculating the average.
- Missing or invalid student information.

Implement appropriate **built-in and user-defined exceptions** and demonstrate the use of try, catch, finally, throw, and throws.

---

5. **E-Commerce Shopping System**

Develop a Java-based **E-Commerce Shopping System** where a customer can add products to a shopping cart and place an order.
The application should handle:

- Attempting to purchase a product that is out of stock.
- Entering an invalid product ID.
- Entering a negative or zero quantity.
- Insufficient account balance during payment.
- Attempting to place an empty order.

==========

DAY_08

Date : 04.09.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Develop a simple banking application that:
    i. Validates account operations
    ii. Uses custom checked exceptions for insufficient balance
    iii. Uses custom unchecked exceptions for invalid input
    iv. Demonstrates exception propagation
    v. Uses throw, throws, try-catch-finally, and multi-catch blocks

==========

DAY_09

Date : 05.09.2026

Time : 5:30 P.M. to 8:00 P.M.

Problem Statement: Generic Student Result Management System

Develop a **Java application for managing student academic results** using **Generics**. The application should demonstrate the use of **generic classes, generic methods, and bounded wildcards (upper and lower bounds)**.

Requirements

Create a `Student<T>` generic class where `T` represents the type of marks obtained by a student.

The class should contain:

- Student ID
- Student name
- A collection of marks of type `T`
- Methods to add marks and display student details.

Tasks

1. Generic Class

- Implement a generic `Student<T>` class.
- Create objects such as:
  - `Student<Integer>` for integer marks.
  - `Student<Double>` for decimal marks.
- Add and display marks for different students.

2. Generic Methods

Create a generic method:

```
static <T> void displayMarks(List<T> marks)
```

The method should display all marks irrespective of their data type.

Also create a generic method to find and return the maximum value from a collection of comparable elements.

IET,PUNE

3. Upper-Bounded Wildcard
Create a method:

```
static void calculateTotal(List<? extends Number> marks)
```

The method should accept lists containing `Integer`, `Double`, `Float`, etc., and calculate the total marks.

Explain why `List<Integer>` and `List<Double>` can be passed to this method.

4. Lower-Bounded Wildcard
Create a method:

```
static void addDefaultMarks(List<? super Integer> marks)
```

The method should add a set of default integer marks to the supplied collection.

Demonstrate its use with:

- `List<Integer>`
- `List<Number>`
- `List<Object>`

Explain why these lists are accepted.

Expected Output

The program should demonstrate:

```
Student ID: 101
Student Name: Rahul
Marks: [78, 85, 91]

Student ID: 102
Student Name: Priya
Marks: [82.5, 88.0, 91.5]

Integer Marks Total: 254
Double Marks Total: 262.0

Maximum Integer Mark: 91
Default marks added successfully.
```

==========

IET, PUNE

DAY_10

Date : 07.09.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Store student details in an ArrayList.
    i. Implement Student Management System:
    ii. Add Student
    iii. Search Student by Roll Number
    iv. Update Student Details
    v. Delete Student
    vi. Display All Students
2. Implement a Task Management System
    i. Use a LinkedList to maintain tasks.
    ii. Add task
    iii. Remove task
    iv. Mark task as completed
    v. Display pending tasks

==========

DAY_11

Date : 08.09.2026

Time : 5:30 P.M. to 8:00 P.M.

1. In Student Merit List System, Generate the following reports:
    i. Sort by Roll Number (Ascending)
    ii. Sort by Name (Ascending)
    iii. Generate a Merit List by Marks (Descending).
    iv. Sort by Department (Ascending) and then Name (Ascending)
    v. Sort by Department (Ascending) and then Marks (Descending)
    vi. Sort by Marks (Descending) and then Roll Number (Ascending)
    vii. Sort by Department (Ascending), Marks (Descending), and then Name (Ascending)

==========

DAY_12

Date : 09.09.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Create custom Functional Interfaces and implement them using Lambda Expressions.
2. Use Predicate, Function, Consumer, and Supplier interfaces in different programs.
3. Implement sorting and searching using Lambda Expressions.

IET PUNE

4. Demonstrate Method References and Constructor References.
5. Create Streams from Collections and Arrays.
6. Perform filtering and transformation operations using Stream API.
7. Implement aggregate operations using reduce(), count(), min(), and max().
8. Group and partition data using Collectors.
9. Develop a mini application to process employee/student records using Stream API.

==========

DAY_13

Date : 10.09.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Create multiple threads using Thread class and Runnable interfaces.
2. Assign same task and different task to multiple threads.
3. Understand sleep, join, yield methods.
4. Implement wait, notify and notifyAll methods.
5. Demonstrate how to share thread local data between multiple threads.
6. Invoke private methods of some other class using reflection.
7. Create multiple threads using anonymous inner classes.
8. Create multiple threads using lambda expressions.

==========

DAY_14

Date : 11.09.2026

Time : 5:30 P.M. to 8:00 P.M.

Student Management System using HashSet

- Store student details in a **HashSet**.

- Implement Student Management System:

  - Add Student
  - Search Student
  - Update Student Details
  - Delete Student
  - Display All Students
  - Check whether a student exists

- Ensure that **duplicate students are not allowed** by properly overriding equals() and hashCode().

- Implement a **Task Management System using HashSet**.

IET, PUNE

● Use a **HashSet** to maintain a unique collection of tasks.

● Support:

- Add Task
- Remove Task
- Mark Task as Completed
- Search Task
- Display Pending Tasks
- Display All Tasks
- Prevent duplicate tasks

Note: For HashMap, Roll Number/Task ID naturally works as a unique key. For HashSet, you should demonstrate uniqueness using equals() and hashCode().

==========

DAY_15

Date : 16.09.2026

Time : 5:30 P.M. to 8:00 P.M.

1. Perform Database CRUD operations using JDBC Classes.

IET, PUNE